Bashir Agro Admin Panel
Phase 6: Dynamic Dashboard & Activity Monitoring Module

Phase 6 Scope

Implement ONLY Dashboard Analytics and Activity Monitoring after Phase 5.

Included:
- Dynamic dashboard cards
- Database-driven statistics
- Today's assignment monitoring
- Recent activity display
- Farmer/task/location status overview
- Monitoring filters
- Query optimization
- Tests

Not included:
- Flutter app
- REST API
- Task submission
- Photo evidence
- GPS validation
- Notifications
- Reports module (Phase 7)

Objective

Replace all static dashboard values and monitoring demo records with real PostgreSQL data.

The dashboard becomes the operational control center for Admin/Owner.

All numbers, tables and statuses must come from Laravel queries and database relationships instead of hard-coded JavaScript values.

Current System After Phase 5

Completed modules:

Farmers
+
Farm Locations
+
Task Templates
+
Task Assignments

Phase 6 uses these existing relationships to create live dashboard information.

Dashboard Architecture

Flow:

PostgreSQL Database

↓

Eloquent Models

↓

Dashboard Controller

↓

Dashboard Service / Query Layer

↓

Blade Dashboard

↓

Admin View

Dashboard Controller

Create:

app/Http/Controllers/Admin/DashboardController.php

Responsibilities:

- Load KPI statistics
- Load today's assignments
- Load recent activities
- Load attention items
- Pass data to Blade view

Do not put complex queries directly inside Blade.

Dashboard KPI Cards

Replace all hard-coded values.

Example cards:

1. Active Farmers

Query:

Farmers where status = active.

2. Active Locations

Query:

FarmLocations where status = active.

3. Active Task Templates

Query:

TaskTemplates where status = active.

4. Today's Assignments

Query:

TaskAssignments where scheduled_date = today.

Dashboard Statistics Requirements

Create database-driven metrics:

Active farmers count

Active locations count

Active task templates count

Today's assignments count

Pending assignments

Completed assignments

Cancelled assignments

Recent activities

Today's Assignment Monitoring

Create monitoring section showing:

Farmer

Task

Location

Scheduled Date

Time

Priority

Status

Assigned By

Example Monitoring Data

Rahim

Task:
Apply Fertilizer

Location:
North Field

Time:
08:00 - 10:00

Priority:
Urgent

Status:
Assigned

Monitoring Controller

Create:

app/Http/Controllers/Admin/MonitoringController.php

Responsibilities:

- Assignment monitoring queries
- Filtering
- Sorting
- Status display

Use eager loading:

```
with([
'farmer',
'taskTemplate',
'location'
])
```

to prevent N+1 queries.

Monitoring Filters

Implement:

Status Filter:

All
Assigned
In Progress

Completed
Cancelled

Priority Filter:

All
Normal
Important
Urgent

Date Filter:

Today
This Week
Custom Date

Search

Add monitoring search.

Search by:

- Farmer name
- Task name
- Location name

Example:

/admin/monitoring?search=rahim

Recent Activity Section

Use existing activity_logs table.

Display:

User

Action

Module

Description

Time

Examples:

Farmer created

Location updated

Task assigned

Activity Query

Use latest activity:

ORDER BY created_at DESC

Limit:

10-20 recent records

Use pagination if full activity page is created later.

Dashboard Blade

Create/update:

resources/views/dashboard/index.blade.php

Requirements:

Preserve:

- Existing green theme
- Cards
- Tables
- Bengali labels
- Responsive layout

Only replace static numbers with Blade variables.

Performance Optimization

Use:

Eager loading

Database indexes

Query aggregation

Caching where necessary

Avoid:

Loop queries

Repeated database calls

Heavy calculations inside Blade

Recommended Services

For cleaner architecture create:

app/Services/DashboardService.php

Responsibilities:

- KPI calculations
- Assignment summaries
- Activity summaries

Controller should call services instead of containing all logic.

Database Queries Examples

Active Farmers:

```
Farmer::where(
'status',
'active'
)->count();
```

Today's Assignments:

```
TaskAssignment::whereDate(
'scheduled_date',
today()
)->count();
```

Recent Activities:

```
ActivityLog::latest()
->limit(10)
->get();
```

Routes

Dashboard:

GET /admin/dashboard

Monitoring:

GET /admin/monitoring

Keep routes protected with Admin authentication middleware.

Security

Use:

- Authentication middleware
- Authorization
- Validation for filters
- Protection against unauthorized data access

Admin should only see permitted system information.

Testing

Create:

tests/Feature/DashboardMonitoringTest.php

Test:

- Dashboard accessible by Admin
- Guest cannot access dashboard
- KPI values match database
- Today's assignments display correctly
- Filters work
- Search works
- Monitoring relationships load correctly
- Recent activity displays

Antigravity Prompt

```
Proceed to PHASE 6 ONLY: Dynamic Dashboard and Activity Monitoring Module.

Use the existing Bashir Agro Laravel project after Phase 5.

Do not redesign the UI.

Do not implement:

- Flutter app
- REST API
- Task submission
- Photo evidence
- GPS validation
- Notifications
- Reports


Goal:

Replace all static dashboard and monitoring data with real PostgreSQL data.


Implement:

- DashboardController
- MonitoringController
```

- Dashboard queries/services
- Dynamic KPI cards
- Today's assignment monitoring
- Recent activity display
- Search
- Filters
- Pagination if required
- Query optimization
- Feature tests

Preserve:

- Bengali UI
- Green agricultural theme
- Existing cards
- Existing tables
- Responsive design

Remove hard-coded statistics and assignment data from JavaScript.

Database must be the source of truth.

After implementation:

Run migrations if required.

Run:

```
php artisan test
```

Run:

```
vendor/bin/pint
```

Report:

- files created
- files modified
- database changes
- routes added
- test results

STOP after Phase 6.

Do not start Phase 7 automatically.